

Assignment 1
Arseniy Uspenskiy
USPARS001

Overview: The program consists of six classes divided across three groups: a record class which is used as the primary data type, two data structures for storing objects of the record classes type, and three application classes designed for letting the user run and interact with the code as well as running an experiment.

PlaceNameEntry: A helper class that stores information about a single place id, name etc. PlaceNameEntry objects are used as the data type that is stored by both PlaceNameArray and PlaceNameBST. The class supports a public getter method for accessing the placeName, which is the primary variable used in comparisons between PlaceNameEntry objects.

PlaceNameArray: stores a collection of PlaceNameEntry objects in an array(ignoring duplicates) while also keeping track of the number of places that are loaded. The array can be created with data or created and then have data loaded afterwards, and supports both searching for a specific PlaceNameEntry object as well as a toString() method for printing to the console.

PlaceSearchArray: Lets the user interact with the PlaceNameArray class supporting loading, searching, printing, a help method and quit.

PlaceNameBST: stores a collection of PlaceNameEntry objects in an BST(ignoring duplicates) while also keeping track of the size of the tree. The tree is created empty and then receives data to be loaded either from a file or one by one, and supports both searching for a specific PlaceNameEntry object as well as a toString() method for printing to the console utilizing in order traversal.

PlaceSearchBST: Lets the user interact with PlaceNameBST, allowing for loading, searching, printing, a help method and quitting.

PlaceExperiment: Utilizes both PlaceNameArray and PlaceNameBST to run an experiment comparing the average number of comparisons made in both the array as well as three unique variations of the BST while searching for a particular place name. Each test consists of 50 search queries each being run on an increasing number of elements from 1000 to 10000 in increments of 1000.

Creative Extensions:

- Print: both PlaceSearchArray and PlaceSearchBST support the additional print method which prints the loaded data to the user with individual PlaceNameEntry objects being separated by \n characters(printed on separate lines).
- Help: supported by PlaceSearchArray and PlaceSearchBST, prints and excerpt from the ReadME file to the user which provides clarification into the functionality of the program that is currently being run.

PlaceSearchArray:

```
lobst@1ArsPC1:~/workspace/uct/csc2001f/Assignment1$ java PlaceSearchArray
What would you like to do? (L)oad (S)earch (P)rint (H)elp (Q)uit
L
Please enter the path to the file: SApplaceNames.csv
Please enter how many places to load: 32
32 places loaded.
What would you like to do? (L)oad (S)earch (P)rint (H)elp (Q)uit
Search
Please enter the name of the place to find: Citrusdal
---Searching for: Citrusdal---
161007 Citrusdal Cederberg Western Cape 7177
Number of Comparisons: 20
What would you like to do? (L)oad (S)earch (P)rint (H)elp (Q)uit
S
Please enter the name of the place to find: Constantia
---Searching for: Constantia---
Place not found in database
Number of Comparisons: 32
What would you like to do? (L)oad (S)earch (P)rint (H)elp (Q)uit
help
- PlaceSearchArray:
Program allows the user to load Place data into an array
and then lets the user to search the array for additional information about a place after providing the name
What would you like to do? (L)oad (S)earch (P)rint (H)elp (Q)uit
load
Please enter the path to the file: SApplaceNames.csv
Please enter how many places to load: 5
5 places loaded.
What would you like to do? (L)oad (S)earch (P)rint (H)elp (Q)uit
P
---Loaded Places---
160001 Kliprand Matzikama Western Cape 205
160002 Matzikama NU Matzikama Western Cape 23881
160003 Rietpoort Matzikama Western Cape 970
160004 Bitterfontein Matzikama Western Cape 986
160005 Nuwerus Matzikama Western Cape 650
What would you like to do? (L)oad (S)earch (P)rint (H)elp (Q)uit
q
```

PlaceSearchBST

```
lobst@1ArsPC1:~/workspace/uct/csc2001f/Assignment1$ java PlaceSearchBST
What would you like to do? (L)oad (S)earch (P)rint (H)elp (Q)uit
Load
Please enter the path to the file: SApplaceNames.csv
Please enter how many places to load: 24
24 places loaded.
What would you like to do? (L)oad (S)earch (P)rint (H)elp (Q)uit
search
Please enter the name of the place to find: Rietpoort
---Searching for: Rietpoort---
160003 Rietpoort Matzikama Western Cape 970
Number of Comparisons: 3
What would you like to do? (L)oad (S)earch (P)rint (H)elp (Q)uit
S
Please enter the name of the place to find: Seattle
---Searching for: Seattle---
Place not found in database
Number of Comparisons: 5
What would you like to do? (L)oad (S)earch (P)rint (H)elp (Q)uit
H
- PlacesSearchBST
Program allows the user to load Place data into a binary search tree
and then lets the user to search for additional information about a place after providing the name
What would you like to do? (L)oad (S)earch (P)rint (H)elp (Q)uit
load
Please enter the path to the file: SApplaceNames.csv
Please enter how many places to load: 3
3 places loaded.
What would you like to do? (L)oad (S)earch (P)rint (H)elp (Q)uit
print
---Loaded Places---
160001 Kliprand Matzikama Western Cape 205
160002 Matzikama NU Matzikama Western Cape 23881
160003 Rietpoort Matzikama Western Cape 970
What would you like to do? (L)oad (S)earch (P)rint (H)elp (Q)uit
Quit
```

Edge Cases: Searching for last or first element, Searching for non-existent element, Searching an empty instance, Printing an empty instance, Searching with a very small N, Searching with a very large N, Loading zero elements.

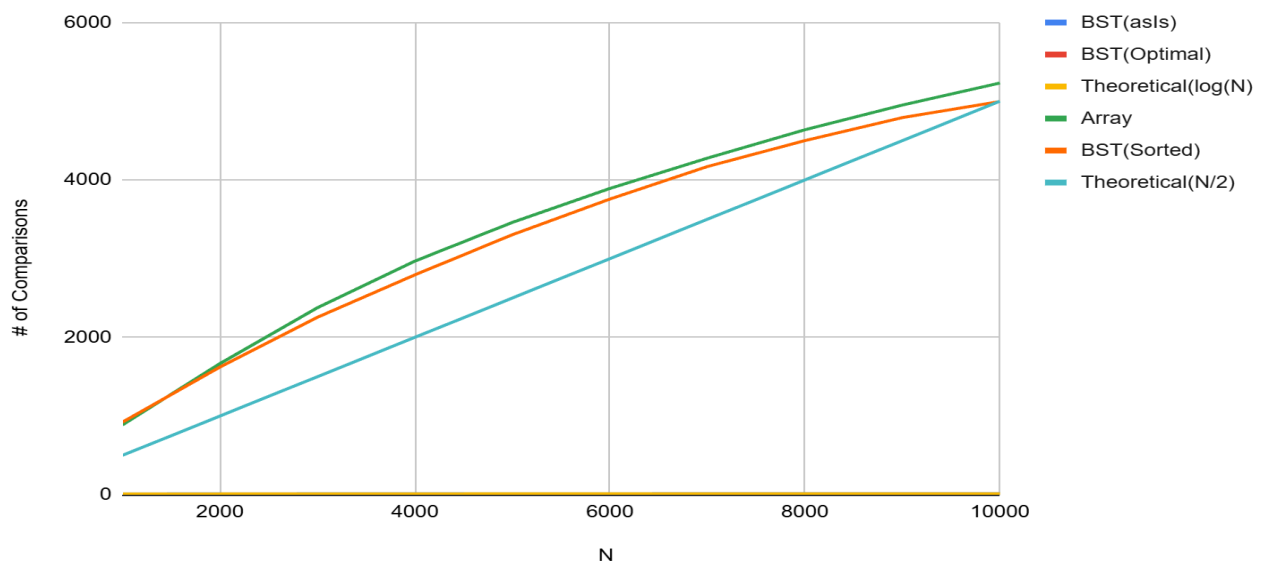
Data Table:

```
lobst@1ArsPC1:~/workspace/uct/csc2001f/Assignment1$ java PlaceExperiment
```

N	Array	BST(as-is)	BST(sorted)	BST(optimal)
1000	887.6	11.6	921.4	9.9
2000	1669.6	12.6	1621.0	10.9
3000	2384.1	13.4	2252.2	11.4
4000	3018.4	13.8	2790.2	11.8
5000	3548.4	14.1	3299.6	12.1
6000	4018.5	14.3	3751.3	12.2
7000	4438.5	14.4	4161.4	12.3
8000	4818.5	14.7	4489.4	12.5
9000	5178.5	14.8	4779.0	12.7
10000	5498.5	14.9	4979.5	12.7

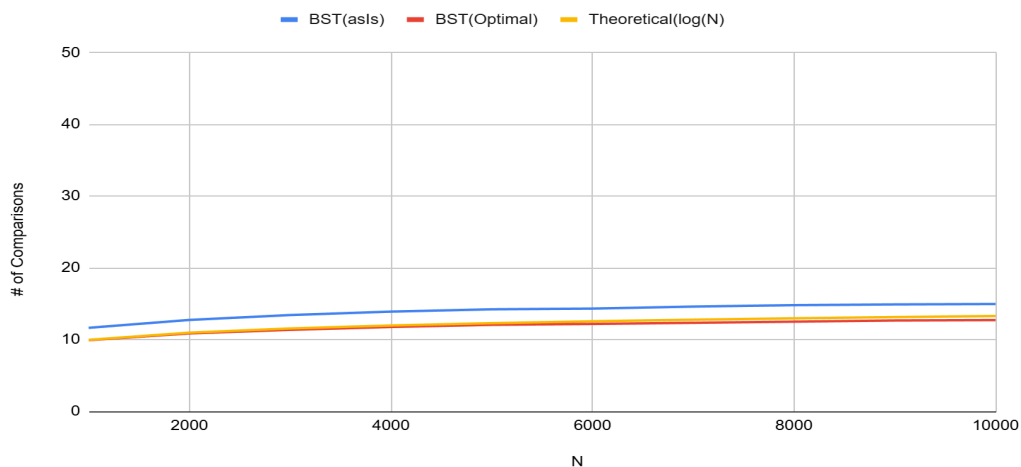
Complete Graph:

BST(asIs), BST(Optimal), Theoretical(log(N), Array, BST(Sorted)...



Graph of only $O(\log(N))$ algorithms:

BST(asIs), BST(Optimal) and Theoretical(log(N))



Method: Using a pre-set file of 50 search queries covering both existing and non-existing places, as well as edge cases such as being at the beginning or end of a loaded set of N data points and including a few randomly selected ones. The search queries were run using an Array, a BST(from base data), a BST(from sorted data), and a BST(from optimally ordered data). The test is run on each data structure 10 times while continuously increasing the number of entries starting from 1000 and ending on 10000 entries, increasing by increments of 1000. The data is gathered into a table and then plotted alongside the theoretical result for an $O(N)$ algorithm and an $O(\log(N))$ algorithm.

Results: Based on both the table and the graph each data structure's results are consistent with the following time complexities. Array : $O(N)$, BST(asIs) : $O(\log(N))$, BST(sorted) : $O(N)$, BST(optimal) : $O(\log(N))$. These values are consistent with the expected theoretical values that were discussed in class.

Conclusion: The data shows a key element of how a Binary Search Tree functions as seen by the fact that two of the BST implementations experienced a complexity consistent with $O(\log(N))$ meanwhile BST(sorted) had a complexity consistent with $O(N)$. This is because the order in which data is inserted into a binary search tree matters. When sorted data is inserted into a BST, every new data point is always placed as the right node of the previous inserted data point creating what is essentially just a linked list. This means that when searching for the largest element that is stored you must first check all of the other loaded elements which is what creates the worst case scenario of $O(N)$ comparisons. Similarly though less noticeable there is a consistent gap of comparisons in between the BST(asIs) and BST(optimal). This is also caused due to the order in which the data is inserted. Over time due to non-optimal inputs the as is tree becomes unbalanced and one side of the tree ends up having a larger height than the other. This results in more search comparisons being required to then find elements on that side accounting for the gap between comparisons amongst the two trees.

AI disclosure: AI was only used towards the end of the assignment to identify potential flaws that may have gone unnoticed in production as well as help with structuring java docs. The tool used was Chat GPT thinking 5.4, no AI code or writing was inserted into the project directly.